

Alternating Turing machine

In computational complexity theory, an **alternating Turing machine** (ATM) is a non-deterministic Turing machine (NTM) with a rule for accepting computations that generalizes the rules used in the definition of the complexity classes NP and co-NP. The concept of an ATM was set forth by Chandra and Stockmeyer^[1] and independently by Kozen^[2] in 1976, with a joint journal publication in 1981.^[3]

Definitions

Informal description

The definition of NP uses the *existential mode* of computation: if *any* choice leads to an accepting state, then the whole computation accepts. The definition of co-NP uses the *universal mode* of computation: only if *all* choices lead to an accepting state does the whole computation accept. An alternating Turing machine (or to be more precise, the definition of acceptance for such a machine) alternates between these modes.

An **alternating Turing machine** is a non-deterministic Turing machine whose states are divided into two sets: **existential states** and **universal states**. An existential state is accepting if some transition leads to an accepting state; a universal state is accepting if every transition leads to an accepting state. (Thus a universal state with no transitions accepts unconditionally; an existential state with no transitions rejects unconditionally). The machine as a whole accepts if the initial state is accepting.

Formal definition

Formally, a (one-tape) **alternating Turing machine** is a 5-tuple $M = (Q, \Gamma, \delta, q_0, g)$ where

- Q is the finite set of states
- Γ is the finite tape alphabet
- $\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$ is called the transition function (L shifts the head left and R shifts the head right)
- $q_0 \in Q$ is the initial state
- $g : Q \rightarrow \{\wedge, \vee, \text{accept}, \text{reject}\}$ specifies the type of each state

If M is in a state $q \in Q$ with $g(q) = \text{accept}$ then that configuration is said to be *accepting*, and if $g(q) = \text{reject}$ the configuration is said to be *rejecting*. A configuration with $g(q) = \wedge$ is said to be accepting if all configurations reachable in one step are accepting, and rejecting if some configuration reachable in one step is rejecting. A configuration with $g(q) = \vee$ is said to be accepting when there exists some configuration reachable in one step that is accepting and rejecting when all configurations reachable in one step are rejecting (this is the type of all states in a classical NTM except the final state). M is said to accept an input string w if the initial configuration of M (the state of M is q_0 , the head is at the left end of the tape, and the tape contains w) is accepting, and to reject if the initial configuration is rejecting.

Note that it is impossible for a configuration to be both accepting and rejecting, however, some configurations may be neither accepting or rejecting, due to the possibility of nonterminating computations.

Resource bounds

When deciding if a configuration of an ATM is accepting or rejecting using the above definition, it is not always necessary to examine all configurations reachable from the current configuration. In particular, an existential configuration can be labelled as accepting if any successor configuration is found to be accepting, and a universal configuration can be labelled as rejecting if any successor configuration is found to be rejecting.

An ATM decides a formal language in time $t(n)$ if, on any input of length n , examining configurations only up to $t(n)$ steps is sufficient to label the initial configuration as accepting or rejecting. An ATM decides a language in space $s(n)$ if examining configurations that do not modify tape cells beyond the $s(n)$ cell from the left is sufficient.

A language that is decided by some ATM in time $c \cdot t(n)$ for some constant $c > 0$ is said to be in the class $\mathbf{ATIME}(t(n))$, and a language decided in space $c \cdot s(n)$ is said to be in the class $\mathbf{ASPACE}(s(n))$.

Example

Perhaps the most natural problem for alternating machines to solve is the quantified Boolean formula problem, which is a generalization of the Boolean satisfiability problem in which each variable can be bound by either an existential or a universal quantifier. The alternating machine branches existentially to try all possible values of an existentially quantified variable and universally to try all possible values of a universally quantified variable, in the left-to-right order in which they are bound. After deciding a value for all quantified variables, the machine accepts if the resulting Boolean formula evaluates to true, and rejects if it evaluates to false. Thus at an existentially quantified variable the machine is accepting if a value can be substituted for the variable that renders the remaining problem satisfiable, and at a universally quantified variable the machine is accepting if any value can be substituted and the remaining problem is satisfiable.

Such a machine decides quantified Boolean formulas in time n^2 and space n .

The Boolean satisfiability problem can be viewed as the special case where all variables are existentially quantified, allowing ordinary nondeterminism, which uses only existential branching, to solve it efficiently.

Complexity classes and comparison to deterministic Turing machines

The following complexity classes are useful to define for ATMs:

- $\mathbf{AP} = \bigcup_{k>0} \mathbf{ATIME}(n^k)$ are the languages decidable in polynomial time
- $\mathbf{APSPACE} = \bigcup_{k>0} \mathbf{ASPACE}(n^k)$ are the languages decidable in polynomial space
- $\mathbf{AEXPTIME} = \bigcup_{k>0} \mathbf{ATIME}(2^{n^k})$ are the languages decidable in exponential time

These are similar to the definitions of $\mathbf{P}$, $\mathbf{PSPACE}$, and $\mathbf{EXPTIME}$, considering the resources used by an ATM rather than a deterministic Turing machine. Chandra, Kozen, and Stockmeyer^[3] proved that, for all $f(n) \geq \log(n)$ and $g(n) \geq \log(n)$:

- $\mathbf{ASPACE}(f(n)) = \bigcup_{c>0} \mathbf{DTIME}(2^{cf(n)}) = \mathbf{DTIME}(2^{O(f(n))})$
- $\mathbf{ATIME}(g(n)) \subseteq \mathbf{DSpace}(g(n))$
- $\mathbf{NSPACE}(g(n)) \subseteq \bigcup_{c>0} \mathbf{ATIME}(c \times g(n)^2),$

In particular:

- $\text{ALOGSPACE} = \text{P}$
- $\text{AP} = \text{PSPACE}$
- $\text{APSPACE} = \text{EXPTIME}$
- $\text{AEXPTIME} = \text{EXPSPACE}$

A more general form of these relationships is expressed by the parallel computation thesis.

Bounded alternation

Definition

An **alternating Turing machine with k alternations** is an alternating Turing machine that switches from an existential to a universal state or vice versa no more than $k-1$ times. (It is an alternating Turing machine whose states are divided into k sets. The states in even-numbered sets are universal and the states in odd-numbered sets are existential (or vice versa). The machine has no transitions between a state in set i and a state in set $j < i$.)

$\text{ATIME}(C, j) = \Sigma_j \text{TIME}(C)$ is the class of languages decidable in time $f \in C$ by a machine beginning in an existential state and alternating at most $j - 1$ times. It is called the j th level of the $\text{TIME}(C)$ hierarchy.

$\text{coATIME}(C, j) = \Pi_j \text{TIME}(C)$ is defined in the same way, but beginning in a universal state; it consists of the complements of the languages in $\text{ATIME}(f, j)$.

$\text{ASPACE}(C, j) = \Sigma_j \text{SPACE}(C)$ is defined similarly for space bounded computation.

Example

Consider the circuit minimization problem: given a circuit A computing a Boolean function f and a number n , determine if there is a circuit with at most n gates that computes the same function f . An alternating Turing machine, with one alternation, starting in an existential state, can solve this problem in polynomial time (by guessing a circuit B with at most n gates, then switching to a universal state, guessing an input, and checking that the output of B on that input matches the output of A on that input).

Collapsing classes

It is said that a hierarchy *collapses* to level j if every language in level $k \geq j$ of the hierarchy is in its level j .

As a corollary of the Immerman–Szelepcsényi theorem, the logarithmic space hierarchy collapses to its first level.^[4] As a corollary the $\text{SPACE}(f)$ hierarchy collapses to its first level when $f = \Omega(\log)$ is space constructible.

Special cases

An alternating Turing machine in polynomial time with k alternations, starting in an existential (respectively, universal) state can decide all the problems in the class Σ_k^P (respectively, Π_k^P).^[5] These classes are sometimes denoted $\Sigma_k P$ and $\Pi_k P$, respectively. See the polynomial hierarchy article for details.

Another special case of time hierarchies is the logarithmic hierarchy.

References

1. Chandra, Ashok K.; Stockmeyer, Larry J. (1976). "Alternation". *Proc. 17th IEEE Symp. on Foundations of Computer Science*. Houston, Texas. pp. 98–108. doi:10.1109/SFCS.1976.4 (<https://doi.org/10.1109/SFCS.1976.4>).
2. Kozen, D. (1976). "On parallelism in Turing machines". *Proc. 17th IEEE Symp. on Foundations of Computer Science*. Houston, Texas. pp. 89–97. doi:10.1109/SFCS.1976.20 (<https://doi.org/10.1109/SFCS.1976.20>). hdl:1813/7056 (<https://hdl.handle.net/1813/7056>).
3. Chandra, Ashok K.; Kozen, Dexter C.; Stockmeyer, Larry J. (1981). "Alternation" (<https://web.archive.org/web/20160412232110/http://users.cis.fiu.edu/~giri/teach/5420/f01/papers/p114-chandra.pdf>) (PDF). *Journal of the ACM*. **28** (1): 114–133. doi:10.1145/322234.322243 (<https://doi.org/10.1145/322234.322243>). S2CID 238863413 (<https://api.semanticscholar.org/CorpusID:238863413>). Archived from the original (<http://users.cis.fiu.edu/~giri/teach/5420/f01/papers/p114-chandra.pdf>) (PDF) on April 12, 2016.
4. Immerman, Neil (1988). "Nondeterministic space is closed under complementation" (<http://www.cs.umass.edu/~immerman/pub/space.pdf>) (PDF). *SIAM Journal on Computing*. **17** (5): 935–938. CiteSeerX 10.1.1.54.5941 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.54.5941>). doi:10.1137/0217058 (<https://doi.org/10.1137/0217058>).
5. Kozen, Dexter (2006). *Theory of Computation* (<https://archive.org/details/theorycomputation00koze>). Springer-Verlag. p. 58 (<https://archive.org/details/theorycomputation00koze/page/n67>). ISBN 9781846282973.

Further reading

- Michael Sipser (2006). *Introduction to the Theory of Computation* (2nd ed.). PWS Publishing. ISBN 978-0-534-95097-2. Section 10.3: Alternation, pp. 380–386.
- Christos Papadimitriou (1993). *Computational Complexity* (1st ed.). Addison Wesley. ISBN 978-0-201-53082-7. Section 16.2: Alternation, pp. 399–401.
- Balcázar, José Luis; Díaz, Josep; Gabarró, Joaquim (1990), Balcázar, José Luis; Díaz, Josep; Gabarró, Joaquim (eds.), "Alternation" (https://link.springer.com/chapter/10.1007/978-3-642-75357-2_4), *Structural Complexity II*, Berlin, Heidelberg: Springer, pp. 63–96, doi:10.1007/978-3-642-75357-2_4 (https://doi.org/10.1007/978-3-642-75357-2_4), ISBN 978-3-642-75357-2, retrieved 2025-05-19
- Bakhadyr Khoussainov; Anil Nerode (2012). *Automata Theory and its Applications* (https://books.google.com/books?id=wR_oBwAAQBAJ). Springer Science & Business Media. ISBN 978-1-4612-0171-7.

This article is issued from Wikipedia. The text is available under [Creative Commons Attribution-Share Alike 4.0](#) unless otherwise noted. Additional terms may apply for the media files.